Software Design Document (SDD) Template

Software design is a process by which the software requirements are translated into a representation of software components, interfaces, and data necessary for the implementation phase. The SDD shows how the software system will be structured to satisfy the requirements. It is the primary reference for code development and, therefore, it must contain all the information required by a programmer to write code. The SDD is performed in two stages. The first is a preliminary design in which the overall system architecture and data architecture is defined. In the second stage, i.e. the detailed design stage, more detailed data structures are defined and algorithms are developed for the defined architecture.

This template is an annotated outline for a software design document adapted from the IEEE Recommended Practice for Software Design Descriptions. The IEEE Recommended Practice for Software Design Descriptions have been reduced in order to simplify this assignment while still retaining the main components and providing a general idea of a project definition report. For your own information, please refer to IEEE Std 10161998[1] for the full IEEE Recommended Practice for Software Design Descriptions.

[1] http://www.cs.concordia.ca/~ormandj/comp354/2003/Project/ieeeSDD.pdf

Team 1

Project 1 – Waterfall

Software Design Document

Name (s): Rahim B (barda002), Arish S (sooma002), Hlina T (tesse044), Adam B (banin010)
Lab Section: 001
Workstation: CSE Lab

Date: 10/17/2025

Table of Contents

1. Introduction 2

1.1 Purpose 2

1.2 Scope 2

1.3 Overview 2

1.4 Reference Material 2

1.5 Definitions and Acronyms 2

2. System Overview 3

3. System Architecture 3

3.1 Architectural Design 3

3.2 Decomposition Description 5

3.3 Design Rationale 6

4. Data Design 6

4.1 Data Description 6

4.2 Data Dictionary 7

5. Component Design 9

6. Human Interface Design 11

6.1 Overview of User Interface 11

6.2 Screen Images 12

6.3 Screen Objects and Actions 12

7. Requirements Matrix 12

1. Introduction

1.1 Purpose

This software design document describes the architecture and system design of the voting system. The purpose of this document is to define how the system will be structured and implemented to fulfill the requirements given. The intended audience includes but is not limited to developers, testers, project managers/owners, and stakeholders.

1.2 Scope

The voting system is an electronic voting processing system that is capable of running two types of voting systems: plurality voting and single transferable vote (STV). The system will read in the ballots, accept user input, calculate the election, display the results, and output an audit file (if required). The goal of this system is to provide a reliable and accurate voting processing system to facilitate fair and accurate voting.

1.3 Overview

Section 1: Introduction - Outlines the purpose of the SDD and scope of the system.

Section 2: System Overview - High-level overview of the system.

Section 3: System Architecture - Overview of the system architecture.

Section 4: Data Design - Provides an overview of the data system architecture

Section 5: Component Design - An overview of the component architecture.

Section 6: Human Interface Design - An overview of the CLI for the user interface.

Section 7: Requirements Matrix - A table showing the mapping from requirements to components

1.4 Reference Material

GitHub Repository: https://github.umn.edu/umn-csci-4801-01-F25/repo-Team1

SRS Document: https://github.umn.edu/umn-csci-4801-01-F25/repo-Team1/tree/main/SRS

1.5 Definitions and Acronyms

CSV: a file type in which data/values are comma-separated.

GitHub: a repository management tool for the voting system

2. System Overview

The system is a fully functioning election system designed to process ballots, count votes, and display election results. The system supports two different types of elections: Single Transferable Voting (STV) and Plurality. Given CSV ballots by an election official, the type of election, and necessary details about the election (e.g., number of candidates), the results will be displayed on the command-line interface and can be accessed later through an audit file. From a design perspective, the system is modular, meaning that different components will handle different tasks such as processing ballots, counting votes, using different election algorithms, and report handling. By splitting each major function into its own component, it becomes possible to extend or modify the system in the future.

3. System Architecture

3.1 Architectural Design

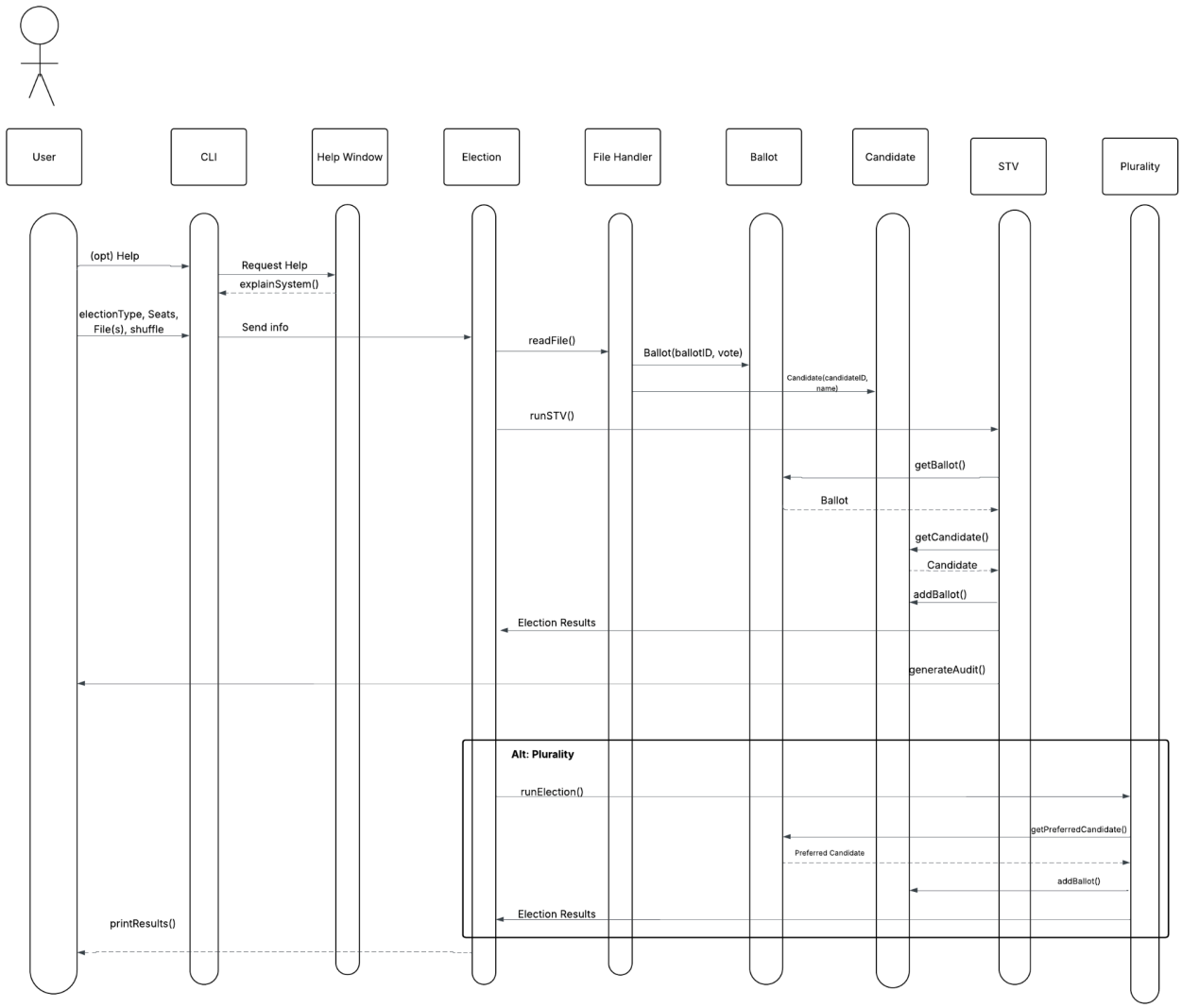
The subsystems have been split up into ballot processing, algorithm processing, and reporting classes. When the election official starts an election, the file handler will take in the file and read in the contents. Next, the ballot class will extract the necessary information, such as the voter's ranking, and pass it along to the respective election algorithm (Plurality or STV). Once the algorithm conducts the election process, the results will go to the reporting system, display the results, and generate an audit report. By splitting up the major processes through Object Oriented Programming, the separate components will work coherently as an election management system. The following UML diagram discusses the specifics of the structure:

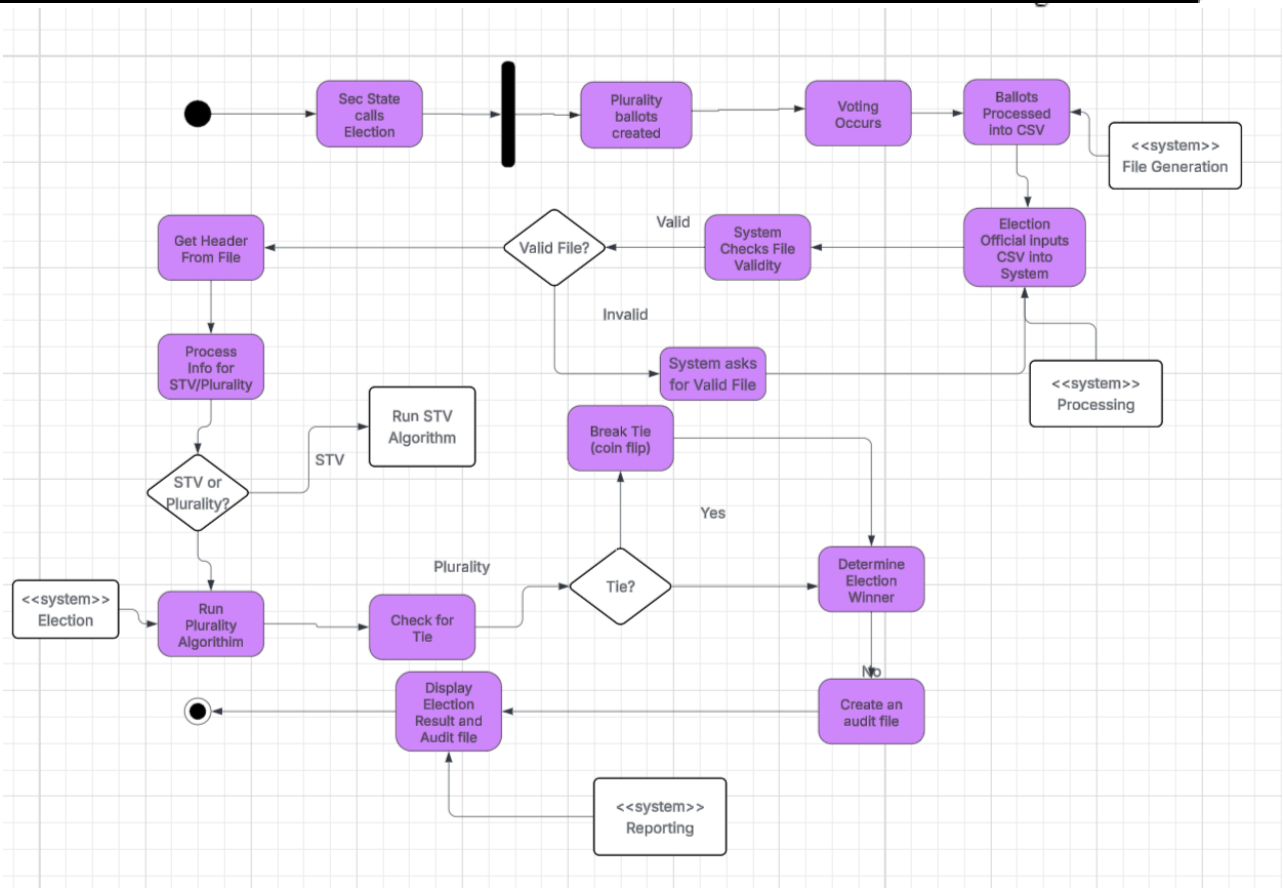
Main
«Static»
HelpWindow
+explainSystem() : void
FileHandler
+ ArrayList csvCandidates
+ ArrayList csvBallots
+ FileHandler()
+ readFile(File BallotFile)
+ getCSVBallots() : ArrayList Array
+ getNumVotes() : int
+ getCandidates() : ArrayList String
«Abstract»
Election
#ArrayList deadBallots
#ArrayList liveBallots
#int availableSeats
#int totalSeats
#ArrayList winners
#ArrayList losers
#ArrayList candidates
#printResults() : void
+runElection() : void
#breakTie(ArrayList tied) : ArrayList winners
+Election(ArrayList candidates, ArrayList ballots, int seats)
Plurality
#printResults() : void
+runElection() : void
#breakTie(ArrayList tied) : ArrayList
+Plurality(ArrayList candidates, ArrayList ballots, int seats)
STV
-Bool shuffle
-Integer droop
#printResults() : void
+runElection() : void
#breakTie(ArrayList tied) : ArrayList
+STV(ArrayList candidates, ArrayList ballots, int seats, bool shuffle)
-generateAudit() : void
-calculateDroop() : void
-shuffle() : void
Candidate
-String name
-String party
-Integer candidateId
-int totalBallots
-ArrayList ballots
+toString() : String
+getCandidateId() : Integer
+getTotalBallots() : int
+dumpBallots() : ArrayList
+addBallot(Ballot b) : void
+Candidate(Integer candidateId, String name)
Ballot
-Iterator prefIterator
-Integer ballotId
-Integer shuffleId
-ArrayList preferences
+getBallotId() : Integer
+setShuffleId() : void
+getShuffleId() : Integer
-iteratePreference() : void
+getPreferredCandidateId() : Integer
+Ballot(Integer ballotId, ArrayList vote)
1
1
*
1
*
*

3.2 Decomposition Description

The STV and Plurality algorithms are responsible for running the main election process (controller). Each algorithm will conduct the voting process according to its rules, determine the winner, and initiate the call to generate an audit report/display results. The flow of the process was designed so that the algorithms wouldn't need to figure out any information; all necessary information would be passed in, making the focus only on conducting their respective election method. The following diagram illustrates the flow in detail:

Sequence Diagram:





3.3 Design Rationale

The design and architecture of the system that is shown in the diagrams, have been chosen to prioritize efficiency, functionality, and scalability. In order to reduce complexity and coupling, each major process has been separated into its own component. For example, file handling is separate from the election algorithm. By separating the functionality, providing each process only the specific data it needs, we reduce the possibility of errors occurring. Furthermore, since we inherit from base/abstract classes such as Election, we can ensure that the system can be easily updated and maintained. Although this is something that seems necessary, the trade-off is that we introduce tight coupling between election, ballot, and candidate, reducing adaptability with scale. Despite this, the benefits regarding updatability, efficiency, and organization outweigh the tradeoffs.

4. Data Design

4.1 Data Description

The voting system processes election data by converting information from the given CSV input into structured memory objects. Candidate and ballot data are read through the FileHandler class, which parses CSV files into Candidate and Ballot objects. These are stored

within an ArrayList as our primary data structure and managed by the Election class and its processes. During the execution of plurality or STV, election logic is preformed on the data structures. Results are then displayed to the end user, and an audit report is generated. The system does not use external data sources/databases. All information is stored on the device. Only CSV and audit report(s) can be external to the system.

4.2 Data Dictionary

4.2.1 Ballot

Name	Data Type	Description
prefIterator	Iterator	Internal iterator over voters' ranked preferences.
ballotId	Integer	Unique ID for the ballot
shuffleID	Integer	Post shuffle ID, used for randomized order when processing votes.
preferences	ArrayList	Ranked candidate IDs in voter preference order.

4.2.2 Candidate

Name	Data Type	Description
shuffleID	int	Candidates shuffle tracking ID: used for ordering and ties.
name	String	Candidates Name
party	String	Candidate Party
candidateID	Integer	Unique candidate ID
totalBallots	int	Count of ballots currently assigned
ballots	ArrayList	Collection of ballots assigned to the candidate.

4.2.3 Election

Name	Data Type	Description
deadBallots	Array List	List of ballots that are no longer active.
liveBallots	ArrayList	List of ballots that are still active
availableSeats	int	Total remaining seats available in the election
totalSeats	int	Total Seats in the election
winners	ArrayList	Order of candidates who have been declared winners
losers	ArrayList	Order of candidates no longer a part of the election
candidates	ArrayList	All candidates in the election

4.2.4 File Handler

Name	Data Type	Description
csvCandidates	ArrayList	Parsed row entry Candidates from CSV
csvBallots	ArrayList	Parsed row entry Ballots from CSV

4.2.5 STV

Name	Data Type	Description
shuffle	Bool	Whether to shuffle the ballots or disable it for development and testing.
Droop	Integer	Droop quota for the election

5. Component Design

FileHandler

- readFile(File name)
    - Take in File with ballot information
    - Save candidates into ArrayList csvCandidates
    - Save ballots into ArrayList csvBallots
- getCSVBallots() : ArrayList<ArrayList<Integer>>
    - Return ArrayList csvBallots
- getCandidates() : ArrayList<String>
    - Return ArrayList csvCandidates
- getNumVotes() : int
    - Return csvBallots size

HelpWindow

- explainSystem()
    - Print all commands for the system to the user interface
    - Wait for user input
    - Return to Main window

Election

- runElection()
    - Abstract
- breakTie(ArrayList tied) : ArrayList
    - Abstract
- printResults()
    - Abstract

Plurality

- runElection()
    - Iterate over all ballots:
        - addBallot(ballot) to the candidateId returned by getPreferredCandidateId
    - Break tie if necessary
    - Print results
- breakTie(ArrayList tied) : ArrayList
    - Assign a random number to each tied candidate within a given range.
    - Choose a number randomly within that range.
    - Choose the candidate closest to that number that doesn't exceed that number, and assign them as a winner.
    - If there are more seats to fill, repeat
    - Return ArrayList of winners
- printResults()

    - Print winners, losers, seats filled, date, and total ballots to screen

STV

- runElection()
    - Shuffle if state variable is true
    - Calculate droop
    - Iterate over all ballots:
        - If candidate is a loser, iterate ballots preference and check again
        - Else add ballot to the candidateId returned by getPreferredCandidateId
        - Remove ballot from live ballots
        - If candidate reaches droop assign as a winner
        - If ballot has no more preferences, add ballot to dead ballots
    - End of round, determine one loser
    - Break tie if necessary
    - Dump ballots from loser and assign ballots as live ballots
    - Repeat from iterating ballots until all candidates are winners or losers
    - If not all seats are filled, break tie
- breakTie(ArrayList tied) : ArrayList
    - If there are losers tied at the end of a round with 0 votes
        - Assign a random number to each tied candidate within a given range.
        - Choose a number randomly within that range.
        - Choose the candidate closest to that number that doesn't exceed that number, and assign them as the loser of the current round.
    - If there are losers tied at the end of a round with at least one vote
        - The one with the highest id number ballot is the loser
    - If all candidates are winners or losers, but there are still open seats
        - The last loser is promoted to a winner.
        - If there are more seats to be filled, repeat
- shuffle(int size) : ArrayList
    - Shuffle numbers from 1 to size
    - Return the list of shuffled numbers
- calculateDroop()
    - Take the ceiling of size of live ballots divided by (total seats + 1)
    - Set the droop as this number
- generateAudit()
    - Open a file for writing named audit{date}{time}.txt
    - Write to the file:
        - The date of the election.
        - The candidates' names and parties.
        - The total number of candidates.
        - The number of seats to be filled by the election.

        - The Elected list, in the order in which the candidates were added to the list.
        - The Non-Elected list, in the order in which candidates were added to the list
- printResults()
    - Print seats filled, date, and total ballots to screen
    - Print winners and losers in the order they won/lost

Candidate

- toString() : String
    - Return String containing candidateId, name, party, and totalBallots
- getCandidateId() : Integer
    - Return candidateId
- getTotalBallots() : int
    - Return totalBallots
- dumpBallots() : ArrayList
    - Create copy of ballots
    - Delete ballots from Candidate
    - Return copy of ballots
- addBallot(Ballot b)
    - Add Ballot b to ballots

Ballot

- getBallotId() : Integer
    - Return ballotId
- setShuffleId(int num)
    - Set shuffleId to number provided in input
- getShuffleId() : Integer
    - Return shuffleId
- iteratePreference()
    - Iterate iterator
- getPreferredCandidateId() : Integer
    - Return Iterator's next if not null
    - Otherwise returns null

Main

- Main()
    - Run command given by user

6. Human Interface Design

6.1 Overview of User Interface

The user runs the program and is prompted for the ballot file and election type. If STV is chosen, they will be asked if they would like to shuffle ballots. They may also type "help" to be given instructions on how to use the program. Once the election is started, no further input will be taken. When completed, the results will be displayed on the screen.

6.2 Screen Images

```
Enter CSV file name: ballots
Enter election type: STV
Would you like to disable shuffle: no
Processing election...
```

```
Election Results
------------------------
Date: 10/16/25
Total candidates: 7
Total seats: 4

Elected:
1. Green, Smith
2. Democrat, Johnson
3. Republican, Ali
4. Independent, Cruz

Non-Elected:
1. Republican, Gordon
2. Democrat, Spieth
3. Green, Woods
```

6.3 Screen Objects and Actions

A discussion of screen objects and actions associated with those objects.

Terminal: will accept predefined commands for running an election or opening the help window.

7. Requirements Matrix

Use Case	Component	Data Structure
UC_001	Main	String type, Scanner scan
UC_002	FileHandler	FileHandler processor, Scanner scan
UC_003	STV	ArrayList<Candidate> winners, int availableSeats, ArrayList<Ballots> ballots

UC_004	STV, Plurality, Election	String type
UC_005	STV, Plurality, Election	ArrayList<Candidate> winners, ArrayList<Candidate> losers, int availableSeats,
UC_006	STV	Boolean type shuffle, Integer droop, ArrayList<Candidate> winners, ArrayList<Candidate> candidates, ArrayList<Candidate> losers, ArrayList<Ballot> deadBallots, ArrayList<Ballot> liveBallots, int availableSeats, int totalSeats
UC_007	Plurality	ArrayList<Candidate> candidates, ArrayList<Candidate> winners, ArrayList<Candidate> losers, int availableSeats, int totalSeats
UC_008	STV	ArrayList<Ballot> liveBallots, Integer shuffleID
UC_009	HelpWindow	Scanner scan, HelpWindow helper, String type